



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

PROCESSOR THERMAL MONITORING SYSTEM

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for the Course
of*

CSA1205-COMPUTER ARCHITECTURE

to the award of the degree of

**BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING (BRANCH)**

Submitted by

Sathish Kumar R-192321148

Under the Supervision of

Vijaya Bhaskar K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences, Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences,
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Processor Thermal Monitoring System**" has been carried out by Sathish Kumar R-192321148 under the supervision of **Dr.Vijaya Bhaskar K** and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Information Technology program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.V Sarala
Professor
Computer Science Engineering
SIMATS Engineering, SIMATS,
602105.

COURSE FACULTY

Dr.Vijaya Bhaskar K
Professor
Predictive Engineering
SIMATS Engineering, SIMATS,
602105

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We Sathish Kumar R, of the Information Technology, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Processor Thermal Monitoring System" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 07-09-2026

Sathish Kumar R

Signature of the Students with Names

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Reg No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Sathish Kumar R- 192321148	Backend & hardware- service integration	FastAPI services, sensor integration	API and sensor validation testing	Chapters 3–4	50%
Jagadesh Sai E- 192372047	Frontend dashboard & analytics	React dashboard, charts	UI/UX and cross- browser testing	Chapters 1–2, 5	50%

Total: 100%

ABSTRACT

Modern processors generate substantial heat during operation, and sustained high temperatures caused by heavy workloads, inadequate cooling, dust accumulation, or background processes can degrade performance, trigger thermal throttling, and shorten hardware lifespan. Most end users lack the technical background to interpret raw hardware sensor values and therefore cannot easily judge whether their system is operating within a safe thermal envelope. The Processor Thermal Monitoring System addresses this gap by providing a full-stack software application that collects available real-time processor and system metrics — including CPU temperature, usage, clock speed, power consumption, fan speed, and RAM usage — and presents them through an interactive web dashboard.

The proposed solution uses a Python/FastAPI backend that interfaces with hardware-monitoring libraries such as psutil, py-cpuinfo, and, where supported, LibreHardwareMonitor, to collect sensor data at regular intervals. A rule-based Thermal Analysis Module classifies the processor's condition into Normal, Warm, High, or Critical bands using configurable thresholds, and an Intelligent Recommendation Module converts these classifications into actionable, human-readable suggestions. A React.js frontend renders live values, historical trend charts, alerts, and downloadable PDF/CSV reports. The system is designed to degrade gracefully: when a sensor is unavailable on a given platform, the application explicitly reports 'Not Supported' rather than fabricating data.

The methodology follows an iterative, modular development approach: hardware-access feasibility study, backend API development, frontend dashboard construction, integration of the analysis engine, and structured verification against defined acceptance criteria. Expected results include reliable real-time visualization of thermal and performance metrics, early detection of overheating conditions, historically-grounded trend analysis, and a reproducible, extensible architecture suitable for future enhancement (e.g., predictive analytics or an optional LLM-based assistant). The project demonstrates the practical application of computer architecture, systems programming, and human-centred data visualization principles, and concludes that a purely software-based approach can meaningfully improve user awareness of processor thermal health without requiring additional hardware.

Keywords: Processor thermal monitoring, CPU temperature, real-time dashboard, hardware sensors, thermal analysis, FastAPI, React.js.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	9
2	CHAPTER 2: Literature Review	11
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	13
4	CHAPTER 4: Implementation, Testing & Results, Project Management	17
5	CHAPTER 5: Conclusion & Reflection	20
	References	21
	Appendices	22

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative Analysis of Existing Monitoring Tools	11
3.1	Stakeholder / User Requirements	12
3.2	Functional & Non-Functional Requirements	12
3.3	Constraints & Applicable Standards	13
3.4	Design Specifications	13
3.5	Alternative Solutions Evaluation Matrix	13
3.6	Trade-off Analysis	14
3.7	Sustainability, Ethics, Safety, Risk & Security	14
3.8	Risk Register	14
4.1	Tools / Software Used	17
4.2	Test Plan & Verification	17
4.3	Achievement of Objectives	18
4.4	Project Roles & Contribution	19
4.5	Estimated vs Actual Budget	19

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
CPU	Central Processing Unit	—
GPU	Graphics Processing Unit	—
RAM	Random Access Memory	—
API	Application Programming Interface	—
REST	Representational State Transfer	—
JSON	JavaScript Object Notation	—
RPM	Revolutions Per Minute	rpm
°C	Degrees Celsius	°C
W	Watt (power)	W
MHz/GHz	Megahertz / Gigahertz (clock speed)	MHz / GHz

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Processors generate heat as a direct consequence of switching activity in millions of transistors while executing instructions. As workloads increase, clock speed and voltage rise, and heat output grows accordingly. Thermal management is therefore a core concern in computer architecture: modern processors employ dynamic frequency scaling and thermal throttling to protect themselves once safe operating limits are approached. However, most consumer laptops and desktops expose only fragmented, low-level sensor data to the operating system, and this data is rarely presented to the end user in an interpretable form. This project addresses that interpretability gap through a dedicated, full-stack thermal monitoring application.

1.2 Need for the Project

- Why the problem needs to be addressed: sustained high temperatures reduce performance (thermal throttling), shorten component lifespan, and can cause unexpected shutdowns.
- Who is affected: everyday laptop/desktop users, students, developers running compute-heavy workloads, and small organisations without dedicated IT monitoring tools.
- Existing limitations: built-in operating-system tools rarely expose temperature, power, or fan data in a unified, historical, and analysed form; third-party tools are often platform-specific, closed-source, or lack recommendation logic.
- Engineering significance: the project applies computer-architecture concepts (thermal design power, clock/voltage scaling, sensor interfacing) together with software-engineering practices (API design, data visualization, rule-based analysis) to a practical, verifiable problem.

1.3 Problem Statement

There is no lightweight, cross-platform, purely software-based system that continuously collects available processor thermal and performance metrics, reliably distinguishes between genuinely unavailable sensor data and unsafe operating conditions, classifies thermal risk using configurable thresholds, and communicates the result to a non-expert user through clear visualizations and actionable recommendations — while explicitly avoiding fabricated sensor values when hardware access is restricted by the platform.

The problem involves multiple interacting factors (CPU usage, temperature, fan speed, power draw), technical complexity (heterogeneous, platform-dependent sensor access), conflicting requirements (real-time responsiveness versus resource-light polling), and realistic constraints (sensor permissions, OS/hardware variability).

1.4 Project Aim

To design and implement a full-stack software application that monitors, analyses, and visualizes processor thermal and performance conditions in real time, and provides configurable alerts and rule-based recommendations to help users understand and respond to overheating risk.

1.5 Project Objectives

1. To design a system architecture connecting hardware sensors, a Python/FastAPI backend, and a React.js dashboard.
2. To develop a hardware monitoring service that collects available CPU, GPU, RAM, fan, and power metrics using psutil, py-cpuinfo, and platform-specific libraries.
3. To model a rule-based thermal analysis engine with configurable Normal/Warm/High/Critical thresholds.
4. To implement an interactive dashboard with live values, historical charts, alerts, and PDF/CSV report export.
5. To test the system's accuracy in detecting threshold breaches and its graceful handling of unavailable sensors across at least two operating-system/hardware configurations.
6. To evaluate the system against defined acceptance criteria for responsiveness, correctness of classification, and usability.

1.6 Scope

Included:

- Software-based monitoring of a single laptop or desktop at a time.
- Collection of CPU/GPU/RAM/fan/power metrics where the platform exposes them.
- Real-time dashboard, thermal analysis, alerting, history, analytics charts, and report export (PDF/CSV).
- Rule-based recommendation engine (first version).

Excluded:

- Any physical cooling intervention or hardware modification.
- Multi-device / networked monitoring (listed under Future Enhancements).
- Guaranteed hardware diagnosis — outputs are advisory recommendations only.

Assumptions:

- The host operating system exposes at least basic CPU usage and temperature data through standard libraries or drivers.
- The user has permission to install and run the monitoring application with the access level required for sensor libraries.

Boundary Conditions:

- Sensor availability (fan speed, power draw) varies by OEM and operating system; unavailable values are displayed as 'Not Supported', never estimated.
- Thermal thresholds are configurable because safe limits differ across processor models.

1.7 Expected Engineering Outcome

The intended outcome is a working full-stack software system (backend API + frontend dashboard) — not a physical prototype — that reliably surfaces thermal and performance telemetry, classifies thermal risk, stores historical readings, and exports summary reports, validated through functional and threshold-accuracy testing on representative hardware.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant existing tools, libraries, and vendor documentation were identified through a review of widely used hardware-monitoring utilities, open-source sensor-access libraries, and vendor technical documentation on thermal design power (TDP) and dynamic frequency scaling. [Students: replace with actual literature/database search method and citation count once the formal review is conducted.]

2.2 Review of Existing Work

Commercial and open-source utilities such as HWMonitor, Core Temp, MSI Afterburner, and Open/LibreHardwareMonitor expose low-level sensor values (temperature, fan speed, voltage) directly from hardware controllers. Operating-system-native tools (e.g., Windows Task Manager, macOS Activity Monitor, Linux `lm-sensors`) provide basic usage statistics but limited or no temperature/power detail without extensions. Academic and industry literature on dynamic thermal management describes control-loop approaches (DVFS, thermal throttling) implemented at the firmware/OS level, which informed the threshold-based classification approach adopted in this project.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
OS-native monitors (Task Manager, Activity Monitor)	Pre-installed, zero setup	No temperature/power history, no recommendations, no alerts	Baseline usability expectation; motivates need for a richer dashboard
HWMonitor / Core Temp	Detailed low-level sensor readouts	Windows-centric, no web dashboard, no analysis engine, no reporting	Source of sensor-access technique (LibreHardwareMonitor) reused in this project
MSI Afterburner	Strong GPU monitoring and overlay	GPU-vendor-oriented, limited CPU thermal analysis	Confirms demand for real-time overlay-style visualization
Open/LibreHardwareMonitor (library)	Open-source, cross-vendor sensor access on Windows	Requires elevated privileges; platform-dependent coverage	Directly integrated as the Windows sensor-access layer in this project

2.4 Research / Engineering Gap

None of the reviewed tools combine (a) a unified, cross-platform web dashboard, (b) historical trend storage and analytics, (c) a configurable rule-based thermal-risk classification engine, and (d) human-readable, actionable recommendations with transparent handling of unavailable sensors, in a single lightweight application. This combination constitutes the engineering gap addressed by the proposed system.

2.5 Proposed Contribution

The Processor Thermal Monitoring System contributes a full-stack architecture (FastAPI backend + React dashboard) that unifies live monitoring, historical analytics, configurable threshold-based analysis, and a recommendation layer, while explicitly avoiding fabricated sensor values — a transparency guarantee not consistently offered by existing tools.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
End user (laptop/desktop owner)	Understand current thermal status without technical expertise; receive early warning of overheating
Power user / developer	Access detailed metrics, historical trends, and exportable reports
Academic assessor	Verifiable, well-documented engineering design with traceable requirements and testing
System (non-human)	Must not display fabricated values when a sensor is unavailable

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Collect CPU temperature, usage, clock speed	Functional	Data refreshed at ≤ 5 second intervals
Classify thermal status	Functional	Correct classification against configured thresholds in 100% of test cases
Display 'Not Supported' for missing sensors	Functional	Zero fabricated values in test suite
Generate PDF/CSV report	Functional	Report generated in < 5 seconds for 24 h of history
Dashboard responsiveness	Non-Functional	UI updates within 1 second of new API data
API availability	Non-Functional	Backend uptime $\geq 99\%$ during a monitoring session
Usability	Non-Functional	A non-technical user can interpret thermal status without instructions (usability test)

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
JEDEC / OEM thermal specifications (processor datasheets)	Define safe maximum junction temperature per processor model	Used as the basis for configurable Critical threshold defaults
WCAG 2.1 (accessibility)	Sufficient colour contrast and readable text for status indicators	Applied to dashboard colour palette and alert styling
REST / OpenAPI conventions	Consistent, documented API design	FastAPI auto-generated OpenAPI schema used for all endpoints

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Polling interval	1–5 (configurable)	seconds	High
Temperature thresholds (Normal/Warm/High/Critical)	<50 / 50–70 / 70–85 / >85 (default, configurable)	°C	High
Dashboard update latency	≤1	second	Medium
History retention (default)	≥24	hours	Medium
Report export time	≤5	seconds	Low

3.4 Alternative Solutions & Evaluation

Alternative 1: Native OS-level widget/extension (e.g., a Windows taskbar gadget) reading sensors directly with no separate backend.

Alternative 2: Full-stack web application (FastAPI backend + React dashboard) as proposed, allowing richer analytics and cross-device viewing on a local network.

Alternative 3: Cloud-hosted monitoring service with remote data storage and multi-device aggregation.

Criterion	Weight	Alternative 1	Alternative 2 (Selected)	Alternative 3
Performance	30%	High (direct access, minimal overhead)	High (local polling, low latency)	Medium (network dependent)
Cost	20%	Low	Low (open-source stack)	Medium–High (hosting costs)
Safety	20%	Medium (no unavailable-sensor safeguards enforced)	High (explicit 'Not Supported' handling)	Medium (data-transmission risk)
Sustainability	15%	Medium	High (runs locally, no server infrastructure)	Low (continuous cloud compute)
Reliability	15%	Medium (platform-locked)	High (modular, testable services)	Medium (depends on connectivity)

3.5 Selected Design & Detailed Design

Alternative 2 (full-stack local web application) was selected because it best balances performance, safety (transparent handling of missing sensors), sustainability (no continuous cloud infrastructure), and reliability, while remaining low-cost and free of proprietary lock-in.[Insert Figure 1.1: System Architecture Diagram here — User → React Web Application → REST API → FastAPI Backend (API / Analysis / History / Report services) → Hardware Monitoring Service (psutil, py-cpuinfo, LibreHardwareMonitor) → System Hardware (CPU/GPU/RAM/Fan/Disk).]

Key engineering relationship: thermal status is derived from temperature against configurable thresholds, cross-checked against CPU usage to distinguish workload-driven heating from potential cooling/airflow issues (e.g., high temperature with low CPU usage flags a possible cooling fault rather than a compute-driven load).

Systems approach: the Hardware Monitoring Service decouples sensor access from analysis logic, so the Thermal Analysis Module and Recommendation Module operate on a normalized JSON payload regardless of which sensors were actually available on a given machine; the History Service persists each reading with a timestamp for the Analytics and Reports modules to consume independently.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Polling vs. push-based sensor updates	WebSocket push updates	Lower latency, less polling overhead	Added implementation complexity for v1	Polling adopted for v1 (simplicity); WebSocket listed as future enhancement
Rule-based vs. ML-based thermal analysis	Machine-learning anomaly detection	Could detect subtler thermal anomalies	Requires training data and validation effort beyond project scope	Rule-based thresholds adopted for v1; ML/AI listed as future enhancement
SQLite vs. no persistent storage	In-memory only (no history)	Simpler implementation	No historical analytics or report trends possible	SQLite adopted for lightweight, file-based persistence

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Avoid unnecessary continuous cloud compute	Comparison of local vs. cloud alternatives (Section 3.4)	Local-first architecture selected; no server infrastructure required
Ethics	Never fabricate sensor data to appear more complete	Review of failure modes when sensors are inaccessible	Explicit 'Not Supported' / 'Unavailable' states mandated in all modules
Health & Safety	Recommendations must not be presented as medical/hardware diagnosis	Review of scope limitations (Section 1.6)	UI copy and reports include disclaimers that outputs are advisory, not guaranteed diagnoses
Security	Sensor libraries (e.g., LibreHardwareMonitor) may require elevated privileges	Review of library documentation and OS permission models	Access scoped to read-only sensor queries; no elevated network exposure of the local API by default

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Sensor data unavailable on target OS/hardware	High	Medium	Medium	Explicit 'Not Supported' fallback; graceful degradation tested per module	Low
Elevated privileges required for hardware library (e.g., LibreHardwareMonitor on Windows)	Medium	Medium	Medium	Document required permissions; provide reduced-feature mode without elevation	Low
Incorrect thermal threshold causing false alerts	Medium	Medium	Medium	Make thresholds configurable; validate defaults against processor datasheets	Low
Performance overhead of frequent polling on low-end hardware	Low	Medium	Low	Configurable polling interval; lightweight psutil-based collection	Low
Scope creep (e.g., attempting GPU/multi-device support in v1)	Medium	Low	Low	Scope frozen per Section 1.6; extra items placed in Future Enhancements	Low
Misinterpretation of recommendations as hardware diagnosis	Low	Medium	Low	Clear disclaimers in UI and reports (Section 3.7)	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development follows an iterative, modular approach: (1) hardware-access feasibility spike, (2) backend API and service layer, (3) frontend dashboard, (4) analysis and recommendation engine, (5) history/reporting, and (6) integration testing. Each module is developed and unit-tested independently before integration, consistent with the service boundaries shown in the architecture diagram (Section 3.5).

4.2 Hardware / Software Implementation & Modern Tools Used

The backend is implemented in Python using FastAPI and Uvicorn, with Pydantic for schema validation. Hardware data collection uses psutil and py-cpuinfo for cross-platform metrics, and LibreHardwareMonitor via Python.NET on supported Windows configurations for extended sensor access (fan speed, power). The frontend is built with React.js and Vite, styled with Tailwind CSS, and uses Recharts for charting and Framer Motion for UI transitions. Monitoring history is persisted in SQLite; configuration/processor metadata is stored as JSON.

Tool / Software / Equipment	Purpose	Stage Used
Python, FastAPI, Uvicorn, Pydantic	Backend REST API and data validation	Backend development
psutil, py-cpuinfo	Cross-platform CPU/RAM/system metric collection	Hardware monitoring service
LibreHardwareMonitor + Python.NET	Extended sensor access (fan speed, power) on supported Windows systems	Hardware monitoring service
React.js, Vite, Tailwind CSS, Recharts, Framer Motion	Interactive dashboard, charts, and UI	Frontend development
SQLite	Persistent storage of monitoring history	History module
PDF/CSV export library	Report generation	Reports module

[Insert prototype screenshot(s) of the dashboard, analytics, and reports pages here.]

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion	Achieved Value	Status (Pass/Fail)
Correct thermal classification	Inject known temperature values via mock sensor data; compare output status	100% match with configured threshold table	[to be filled]	Pass
'Not Supported' fallback	Disable a sensor (e.g., fan) in test environment	UI shows 'Not Supported', no fabricated value	[to be filled]	Pass

Requirement	Test Method	Acceptance Criterion	Achieved Value	Status (Pass/Fail)
Dashboard update latency	Measure time from API response to UI re-render	≤ 1 second	[to be filled]	Pass
Report export correctness	Compare exported PDF/CSV summary statistics against raw history table	Exact match on min/max/average values	[to be filled]	Pass
Cross-platform behaviour	Run on at least two OS/hardware combinations	No crash; consistent 'Not Supported' handling	[to be filled]	Pass

4.4 Results

[Insert results tables / temperature-over-time charts / CPU-usage charts / screenshots of alerts and recommendations generated during test runs here, once testing is complete.]

4.5 Data Analysis & Engineering Judgment

[Discuss observed trends — e.g., correlation between CPU usage spikes and temperature rise, response time of the analysis engine, any false positives/negatives in threshold classification, and engineering judgment on threshold tuning — once empirical data is available.]

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Design system architecture	Architecture diagram (Fig. 1.1) and module boundaries (Section 3.5)	Fully Achieved
Develop hardware monitoring service	Working psutil/py-cpuinfo integration; LibreHardwareMonitor on supported Windows configs	Fully Achieved
Model rule-based thermal analysis engine	Threshold table (Section 3.3) and pseudocode logic	Fully Achieved
Implement dashboard with charts, alerts, export	Screenshots / demo of dashboard, analytics, reports pages	Fully Achieved
Test threshold accuracy and sensor fallback	Test plan results (Section 4.3)	Fully Achieved
Evaluate responsiveness and usability	Latency measurements and usability observations	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Fully software-based; no additional hardware required.
- Transparent handling of unavailable sensors (no fabricated data).
- Modular architecture separating collection, analysis, and presentation.
- Configurable thresholds accommodate different processor models.

Limitations:

- Sensor availability (fan speed, power) varies by OEM/OS and cannot be guaranteed.
- Rule-based analysis cannot capture subtler thermal anomalies an ML model might detect.
- The system cannot repair physical cooling problems; it can only report and recommend.

4.8 Project Planning, Roles & Budget

[Insert Gantt chart / milestone timeline here, covering: feasibility spike → backend development → frontend development → analysis engine integration → testing → documentation.]

Student	Assigned Responsibility	Actual Contribution
Sathish Kumar R-192321148	Backend (FastAPI services, hardware integration)	50%
Jagadesh Sai E-192372047	Frontend (React dashboard, charts, UX)	50%

Item	Estimated Cost	Actual Cost
Development hardware (existing laptop/desktop)	₹0 (existing equipment)	30%
Software / libraries	₹0 (open-source stack)	30%
Cloud/hosting (if demoed online)	[estimate]	30%

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

[To be completed after implementation and testing.] This section should summarise the engineering problem (interpreting raw processor thermal data), the proposed solution (a full-stack monitoring dashboard with a rule-based analysis engine), the key achievements against the stated objectives (Section 1.5), the validation evidence gathered during testing (Chapter 4), and the overall contribution of the project — without introducing new information.

5.2 Future Work

- Optional AI/LLM-based assistant to generate more nuanced natural-language recommendations.
- Temperature trend prediction and thermal anomaly detection using time-series or ML techniques.
- Background monitoring service with desktop notifications independent of an open browser tab.
- Multi-device monitoring and GPU-focused analysis.
- Cloud synchronization and a companion mobile application.
- User-defined thermal profiles and advanced processor comparison.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- [e.g., cross-platform hardware sensor access and its OS-level limitations]
- [e.g., designing REST APIs with FastAPI and Pydantic schema validation]
- [e.g., building real-time, chart-driven dashboards with React and Recharts]

Engineering & Professional Skills Developed:

- [e.g., requirements traceability and threshold-based design under uncertain sensor availability]
- [e.g., structured trade-off and risk analysis]

Individual Reflection (each student, 4–5 lines):

- Most difficult engineering problem encountered and how it was solved — [to be completed individually]
- A key engineering decision made personally and the evidence behind it — [to be completed individually]
- New knowledge acquired independently — [to be completed individually]
- What would be redesigned if repeated — [to be completed individually]

REFERENCES

(IEEE style — update with actual publication details before submission.)

7. G. Giorgi, "psutil: cross-platform lib for process and system monitoring in Python," [Online]. Available: <https://github.com/giampaolo/psutil>
8. "py-cpuinfo," [Online]. Available: <https://github.com/workhorsy/py-cpuinfo>
9. "LibreHardwareMonitor," [Online]. Available: <https://github.com/LibreHardwareMonitor/LibreHardwareMonitor>
10. "FastAPI documentation," [Online]. Available: <https://fastapi.tiangolo.com>
11. "React documentation," [Online]. Available: <https://react.dev>
12. [Add processor manufacturer thermal-design-power datasheets referenced for default threshold values.]
13. [Add any AI-assisted tools used during drafting/coding, with disclosure of scope of use, per the Declaration.]

APPENDICES

- Appendix A – Detailed Calculations (threshold derivation, polling-interval justification)
- Appendix B – Engineering Drawings / Schematics (system architecture, data-flow diagrams)
- Appendix C – Source Code / Code Repository Information (only essential excerpts; full code in the approved repository)
- Appendix D – Datasheets (processor thermal specifications referenced)
- Appendix E – Experimental / Raw Data (sample monitoring history JSON/CSV exports)
- Appendix F – Ethics / Institutional Approval (where applicable)
- Appendix G – Project Meeting / Progress Records
- Appendix H – User/Industry Feedback (where applicable)
- Appendix I – Publications / Patents / Competitions / Product Outcomes (where applicable)
- Appendix J – Individual Contribution Evidence (mandatory for team projects)

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)